Soma Kesara

Architecture Learning Guide

For Engineers growing into Architecture

A curated, opinionated path from strong engineer to credible systems architect. Built on the right books, the certification, and the right habits.

Soma Kesara

ARCHITECTURAL JUDGEMENT

The Honest Truth About Becoming an Architect

> No single certification makes you an architect. Architectural judgement is built through reading the right material, applying it to real systems, and reflecting on decisions.

This is not a comprehensive reading list. It is a deliberately short, high-signal path designed to build architectural judgement as fast as possible.

Most engineers chase titles, certifications, or frameworks — and end up with credentials that don't translate to better decisions. This guide gives you the **shortest credible path** to genuine architectural thinking: a thinking model, real-world grounding, platform depth, and the habits that keep you honest.

WHERE TO START

1. Build the Thinking Model First

Resource

Software Architecture in Practice, 4th Ed. **Bass, Clements & Kazman — SEI CMU**

Why This Book First?

Most engineers jump straight to patterns and platforms. That produces architects who can build things but cannot reason about *why* one approach is better than another in a given context.

What You Will Learn

- → Think in **quality attributes** — availability, reliability, scalability, resiliency
- → Make **explicit trade-offs** rather than intuitive guesses
- → Build the **conceptual foundation** that every other resource sits on

2. Understand How Real Systems Are Designed

CORE READING

Resource

Designing Data-Intensive Applications — Martin Kleppmann

This is the best practical book on how application and data flow decisions are made in production systems. It covers **consistency, reliability, and scalability** in a way that is grounded in real engineering — not theory.

How to Read It

Read it **alongside the SEI CMU book**. The two books complement each other directly — one gives you the thinking model, the other gives you the practical vocabulary to apply it. You will start seeing architecture in the systems you work on every day.

- Chapters 1, 5, 7, and 11 are especially high-leverage
- Grounded in production systems, not academic theory
- Widely respected across the industry

3. Learn What Reliable Systems Actually Demand

FREE RESOURCE

Resource: *Site Reliability Engineering* — Google SRE Team · *Free at sre.google/books*

SLOs & Error Budgets

Learn to define service level objectives precisely and manage reliability as a quantified engineering concern — not a vague goal.

Graceful Degradation

Design systems that fail partially and recover — not ones that collapse completely under unexpected load or failures.

Architecture, Not Operations

Availability and reliability are **architectural decisions** made at design time, not operational patches applied after launch.

Written by people running some of the most complex distributed systems in the world. Free, practical, and directly applicable to your design decisions.

4. Ground Your Principles in a Real Platform

CERTIFICATION

AWS Solutions Architect Professional

Exam: SAP-C02

Go straight to **Professional level**. Associate undersells mid-level capability — the questions are too shallow to challenge your thinking or build real trade-off skills.

AWS is the dominant cloud platform. This cert gives you both the depth and the market recognition that matters.

Why Professional, Not Associate?

The Professional exam forces **scenario-based trade-off reasoning** — exactly the skill the SEI CMU thinking model is training. The two reinforce each other directly.

Pair With: AWS Well-Architected Framework

- Free from AWS — no excuse not to read it
- Focus on **Reliability** and **Security** pillars first
- Apply it to real systems, not just study material

5. Stay Away from the Ivory Tower Habit

OPTIONAL

Resource

Building Evolutionary Architectures
Ford, Parsons & Kua — ThoughtWorks

The biggest risk for any architect is becoming **disconnected from delivery** i.e. designing in isolation, producing documents nobody reads, and calling it architecture.

What It Teaches

- Treat architecture as a **continuous practice** embedded in the team
- Validate architecture through **fitness functions**, not documents
- Evolve systems as requirements change — not as a big-bang redesign

Recommended once you have solid footing in resources 1–4. Not critical on day one, but valuable before you start leading architecture decisions in a team.

6. Make Your Decisions Visible

OPTIONAL

Practice: Architecture Decision Records (ADRs) + C4 Diagrams

Architecture Decision Records

ADRs force you to articulate *why* a decision was made — not just what was decided. This is the core skill of an architect. Start writing them for every meaningful decision from day one. Future-you and your team will thank you.

C4 Diagrams

C4 gives teams a **shared visual language** for discussing systems without drowning in unnecessary detail. Use C4 to communicate clearly with engineers and stakeholders — not to impress anyone with complexity.

Highly recommended as you grow into leading design conversations. Not a blocker for getting started, but adopt both early to build the habit.

What About Data Flow & Information Architecture?

OPTIONAL

Understanding how data moves across a system — **event streaming, consistency models, data ownership boundaries** — is valuable context for an application architect. However, it is a deep enough topic to be its own specialism.

Start here only once the core resources above are well understood, and keep your focus on **flow and integration patterns** rather than data modelling or analytics.

Recommended Entry Point

Designing Data-Intensive Applications — Kleppmann covers this well without going too deep. Focus on **Chapters 1, 5, 7, and 11** specifically for data flow and cross-service integration.

Pursue This When

- Your role involves designing systems with complex data flows
- Cross-service integration at scale is a core concern
- You have solid footing in all four core resources

Don't Chase

- Data modelling depth
- Analytics or warehousing
- This specialism before the foundation is solid

What About TOGAF?

OPTIONAL — PROBABLY NOT YET

The Short Answer: Skip It For Now

TOGAF is **process-heavy and delivery-light**. It was designed for large enterprise governance contexts and teaches framework compliance over genuine architectural thinking.

At this stage of your growth, TOGAF will not make you a better architect. It will make you better at filling in TOGAF templates, which is not the same thing.

When to Revisit TOGAF

- Organization explicitly uses TOGAF as a governance standard
- You move into an **enterprise architecture** or large consulting role
- You are operating in a highly regulated or government context

Bottom Line

Come back to it only when the context demands it. The four core resources will serve you far better right now.

The One Thing That Matters Most

> Read with a real system in mind. Every concept in every book above should prompt the question — *how does this apply to something I am building right now?*

Architects who apply ideas to real delivery problems grow fast. Architects who read without applying drift toward theory — and theory without application is not architecture, it is just vocabulary.

1

Build Things

Apply concepts to real systems you are working on — not toy examples.

2

Make Decisions

Own architectural choices and defend the trade-offs explicitly.

3

Write Them Down

Use ADRs to make your reasoning visible and reviewable.

4

Revisit Them

Good architects learn from decisions — especially the ones that didn't age well.